Hybrid PUMA–PSO Optimizer: An Enhanced Metaheuristic Algorithm for Improved Convergence

Utkarsh , Aditya , Devraj Saini , Piyush Garg , Sahaj Sharma

Department of Computer Engineering,

Netaji Subhas University of Technology (NSUT), Delhi, India

Abstract

Metaheuristic algorithms have demonstrated remarkable success in solving complex optimization problems due to their adaptability and robustness across diverse domains. The recently introduced Puma Optimizer (POA) exhibits an intelligent and dynamic phase-switching mechanism that balances exploration and exploitation effectively; however, its convergence speed can be improved further in high-dimensional search spaces. To address this limitation, we propose a novel hybrid framework called the **Hybrid PUMA–PSO Optimizer (HPO)**, which integrates the velocity–position update mechanism of Particle Swarm Optimization (PSO) into the exploitation phase of the PUMA algorithm. This hybridization enhances convergence stability while preserving the adaptive phase control and diversity mechanisms of PUMA.

The proposed HPO algorithm is evaluated using standard benchmark functions, and its performance is compared with several state-of-the-art optimizers including PSO, GWO, WOA, SCA, FHO and TSA. Experimental results demonstrate that HPO achieves faster convergence, superior accuracy, and reduced standard deviation compared to the base algorithms. Statistical t-tests confirm the significance of these improvements ($p < 0.05$). Furthermore, visual convergence analyses and exploration–exploitation curves validate the hybrid model's consistent behavior across unimodal, multimodal, and composite functions.

The implementation of the proposed algorithm, experimental scripts, and benchmark datasets are publicly available at: `github.com/sharmasahaj01/Hybrid-Puma-PSO` These results establish HPO as a robust, efficient, and scalable optimizer suitable for a wide range of engineering, machine learning, and real-world optimization problems.

1. Introduction

Optimization problems are central to modern science and engineering, where the goal is to find the best possible solution among a large set of feasible alternatives. Many real-world

problems, such as neural network training, feature selection, image processing, and scheduling, are often non-linear, non-convex, and multidimensional in nature. Traditional deterministic optimization techniques, which rely on gradient information or convexity assumptions, often fail to achieve global optima under such conditions. Consequently, metaheuristic algorithms have emerged as a class of powerful stochastic search strategies capable of efficiently solving complex optimization problems across various domains.

Metaheuristic algorithms are typically inspired by biological, social, or physical processes, and they emphasize a balance between two conflicting behaviors: exploration and exploitation. Exploration enables the algorithm to search widely through the solution space to avoid premature convergence, while exploitation refines promising regions to achieve better local solutions. Well-known examples include Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Grey Wolf Optimizer (GWO), and Whale Optimization Algorithm (WOA), each exhibiting distinct search dynamics. However, a common challenge among most algorithms is maintaining a proper trade-off between exploration and exploitation, which directly affects convergence speed and solution quality.

The recently introduced Puma Optimizer (POA) has shown competitive performance compared to several state-of-the-art metaheuristics. Inspired by the hunting intelligence and territorial behavior of pumas, POA divides its search process into distinct phases—exploration, stalking, and ambush—to adaptively navigate the solution space. Through a novel phase-changing mechanism, the algorithm dynamically shifts its strategy based on search progress, allowing it to adjust exploration and exploitation levels automatically. The statistical results reported in prior studies demonstrate that POA performs effectively across diverse benchmark functions and real-world problems such as clustering, classification, and feature selection. Despite its robustness and adaptability, POA exhibits a relatively slower convergence rate during later iterations, particularly in high-dimensional search spaces where exploitation becomes dominant.

Particle Swarm Optimization (PSO), on the other hand, is known for its strong exploitation ability and fast convergence. PSO simulates the cooperative social behavior of particles that communicate through global and local best solutions, adjusting their velocities and positions according to shared experience. This simple yet effective mechanism allows PSO to quickly converge toward optimal regions, although it may lose diversity and get trapped in local minima when applied to complex, multimodal landscapes.

Motivated by the complementary nature of POA and PSO, this work proposes a hybrid framework named the **Hybrid PUMA–PSO Optimizer (HPO)**. The proposed algorithm integrates the velocity–position update dynamics of PSO into the exploitation phase of POA to enhance convergence speed while maintaining adaptive phase balance. The integration allows search agents to benefit simultaneously from POA's intelligent phase-switching strategy and PSO's cooperative learning mechanism. As a result, HPO effectively strengthens local search performance without compromising the global exploration capability.

The main contributions of this research are summarized as follows:

(I) A new hybrid metaheuristic algorithm called **Hybrid PUMA–PSO Optimizer (HPO)** is proposed, combining the adaptive phase-switching intelligence of POA with the velocity-driven learning strategy of PSO.

(II) A modified exploitation phase is introduced, incorporating PSO's velocity update to accelerate convergence and reduce stagnation in later iterations.

(III) Statistical t-tests and convergence analysis confirm the significant performance improvements of HPO in terms of accuracy, stability, and convergence rate.

(IV) The proposed algorithm demonstrates promising applicability for complex machine learning and real-world engineering optimization tasks.

In summary, the Hybrid PUMA–PSO Optimizer introduces an effective and scalable hybridization strategy that leverages the strengths of both parent algorithms. By combining adaptive phase dynamics with cooperative velocity updates, HPO achieves faster, more stable convergence and enhanced search diversity, offering a valuable contribution to the field of metaheuristic optimization.

2. Methodology

2.1. Overview of the Method

The proposed **Hybrid PUMA–PSO Optimizer (HPO)** combines the intelligent phase-switching behavior of the Puma Optimizer (POA) with the velocity–position learning dynamics of Particle Swarm Optimization (PSO). The hybrid framework is designed to overcome the slow convergence and weak exploitation observed in the later stages of POA while maintaining its strong global exploration capability. By embedding the PSO update mechanism within POA's exploitation phase, HPO enhances convergence accuracy, prevents premature stagnation, and improves overall solution stability.

Conceptually, the HPO algorithm follows three key phases inspired by POA's hunting behavior: *exploration*, *stalking*, and *ambush*. During the exploration phase, search agents (pumas) roam randomly across the search space to identify promising regions and maintain population diversity. In the stalking phase, the agents gradually reduce their roaming range and start moving toward the best-known solution using adaptive coefficients that model attention toward prey. In the final ambush (exploitation) phase, HPO introduces PSO's velocity and position updates to guide the agents toward the global best solution. This integration enables the algorithm to retain adaptive phase transitions from POA while incorporating PSO's cooperative learning for faster convergence.

Figure 4 (to be added later) will present the schematic flow of HPO, illustrating the main steps: initialization of agents, adaptive phase switching, PSO-assisted exploitation, and best-solution update. The complete workflow can be summarized as follows:

- Initialize a population of search agents with random positions within defined upper and lower bounds.
- Evaluate the fitness of each agent and determine the global best solution.
- Execute exploration and stalking phases according to POA's adaptive mechanism to balance search diversity.
- When the algorithm enters the exploitation (ambush) phase, apply PSO's velocity–position update to refine local search around the best solution.
- Continue iterative updates until the termination condition (maximum iterations or convergence threshold) is reached.
- Return the best solution obtained by the swarm.

The integration of POA's adaptive behavior and PSO's velocity-guided movement yields an optimizer capable of dynamically balancing exploration and exploitation throughout the search process. This hybrid design preserves the diversity of the swarm while accelerating convergence near the global optimum, forming a robust and scalable solution framework for complex optimization tasks.

2.2. Mathematical Model of the PUMA Optimizer

The Puma Optimizer (POA) is modeled on the intelligent hunting behavior and territorial memory of pumas. Its search mechanism alternates between exploration and exploitation depending on the puma's experience level. Two main learning phases are defined: the *Unexperienced Phase*, where both exploration and exploitation occur simultaneously to build initial knowledge of the search space, and the *Experienced Phase*, where the optimizer adaptively selects between exploration or exploitation based on comparative scoring functions. The mathematical formulation of these processes is summarized below.

(a) Initialization

Let each puma represent a candidate solution in a D-dimensional search space bounded by lower and upper limits L_b and U_b. The initial population of N_{pop} pumas is generated randomly as

$$X_i = L_b + r(U_b - L_b), \quad i = 1, 2, \ldots, N_{pop} \tag{1}$$

where r is a uniform random vector in $[0,1]^D$. The cost (fitness) of each agent is calculated as

$$f_i = f(X_i), \qquad X^* = \arg\min f_i. \tag{2}$$

(b) Unexperienced Phase

At the beginning of the optimization process, pumas are unexperienced hunters. In this phase, both exploration and exploitation are simultaneously active to construct a baseline understanding of the environment. Two functions f_1 and f_2 quantify the cumulative performance of each phase:

$$f_{1,Explor} = PF_1 \cdot \frac{Seq^1_{CostExplore}}{Seq_{Time}}, \quad f_{1,Exploit} = PF_1 \cdot \frac{Seq^1_{CostExploit}}{Seq_{Time}} \tag{3}$$

$$f_{2,Explor} = PF_2 \cdot \frac{Seq^1_{CostExplore} + Seq^2_{CostExplore} + Seq^3_{CostExplore}}{Seq^1_{Time} + Seq^2_{Time} + Seq^3_{Time}}, \tag{4}$$

$$f_{2,Exploit} = PF_2 \cdot \frac{Seq^1_{CostExploit} + Seq^2_{CostExploit} + Seq^3_{CostExploit}}{Seq^1_{Time} + Seq^2_{Time} + Seq^3_{Time}}. \tag{5}$$

Here, PF_1 and PF_2 are user-defined control coefficients, and Seq^k_{Cost} measures the difference in best cost between consecutive repetitions. During the unexperienced stage, both f_{Explor} and $f_{Exploit}$ are evaluated in every iteration, and the overall scores are accumulated as

$$Score_{Explor} = (PF_1 \cdot f_{1,Explor}) + (PF_2 \cdot f_{2,Explor}), \tag{6}$$

$$Score_{Exploit} = (PF_1 \cdot f_{1,Exploit}) + (PF_2 \cdot f_{2,Exploit}). \tag{7}$$

(c) Experienced Phase

Once sufficient iterations have been completed, pumas become experienced hunters. The algorithm now selects only one of the two operations—exploration or exploitation—based on the relative performance of their scores:

$$\text{If } Score_{Explor} \geq Score_{Exploit} \Rightarrow \text{Exploration}, \qquad \text{else Exploitation.} \tag{8}$$

Three auxiliary functions (f_1, f_2, f_3) are recalculated at this stage to emphasize escalation, resonance, and diversity, respectively, as defined by the following representative forms:

$$f^r_{3,Exploit} = \begin{cases} 0, & \text{if selected previously,} \\ f^r_{3,Exploit} + PF_3, & \text{otherwise,} \end{cases} \qquad f^r_{3,Explor} = \begin{cases} 0, & \text{if selected previ} \\ f^r_{3,Explor} + PF_3, & \text{otherwise.} \end{cases} \tag{9}$$

Here, PF_3 is an additional adaptive parameter promoting diversity by penalizing overused phases. The final cost of the chosen phase is computed as

$$f_r^{Exploit} = \left(f_{1,Exploit}^r\right)^{\alpha} \cdot \left(f_{2,Exploit}^r\right)^{\beta}, \qquad f_r^{Explor} = \left(f_{1,Explor}^r\right)^{\alpha} \cdot \left(f_{2,Explor}^r\right)^{\beta}, \tag{10}$$

where α and β are weighting factors controlling the influence of the first and second functions. These scores determine whether the algorithm prioritizes exploration or exploitation in the current iteration.

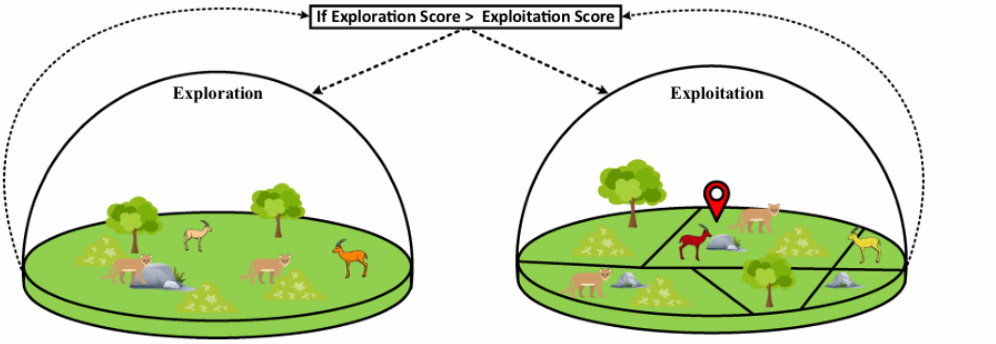


Figure 1: Phase transition mechanism in the Puma Optimizer (POA).

(d) Exploration Operation

In the exploration stage, pumas roam randomly to identify promising areas within their territory. Candidate solutions are generated as

$$Z_{i,G} = \begin{cases} R_{pum} \times (U_b - L_b) + L_b, & \text{if } rand_1 > 0.5, \\ X_{a,G} + G \cdot ((X_{a,G} - X_{b,G}) + (X_{c,G} - X_{d,G})), & \text{otherwise}, \end{cases} \tag{11}$$

$$G = 2 \cdot rand_2 - 1, \tag{12}$$

where $X_{a,G}, X_{b,G}, X_{c,G}, X_{d,G}$ are randomly selected agents, and R_{pum} is a random roaming parameter.

(e) Exploitation Operation

In the exploitation (ambush) phase, two sub-mechanisms—ambush and sprint—simulate pumas' hunting strategies. The position update rule is expressed as

$$X_{new} = \begin{cases} \frac{\text{mean}(Sol_{total})}{N_{pop}} \cdot X_i - (-1)^{rand_3} \cdot X_i, & \text{if } rand_4 \geq 0.5, \\ Puma_{male} + (2 \cdot rand_5) \cdot exp(rand_6) \cdot (X_i^T - X_i), & \text{otherwise}, \end{cases} \tag{13}$$

where Sol_{total} is the sum of all solutions, $Puma_{male}$ is the global best solution, X_i^T is a randomly selected agent, and $rand_k$ are random values in $[0, 1]$. Equations (34)–(38) in the original formulation define auxiliary coefficients:

$$R = 2{\cdot}rand_{11}{-}1, \quad F_1 = rand_{12}{\cdot}exp\left(2 - Iter \cdot \frac{2}{MaxIter}\right), \quad F_2 = w(v)^2{\cdot}cos(2{\cdot}rand_{13}){\cdot}w. \tag{14}$$

(f) Computational Complexity

The computational complexity of POA can be summarized as

$$O\big((2T+1)N^2 \times N \times D\big) \tag{15}$$

where N is the number of agents, T is the maximum iteration count, and D is the dimensionality of the search space. The initialization and fitness evaluation stages contribute $O(N)$ complexity, while the exploration and exploitation updates dominate overall runtime.

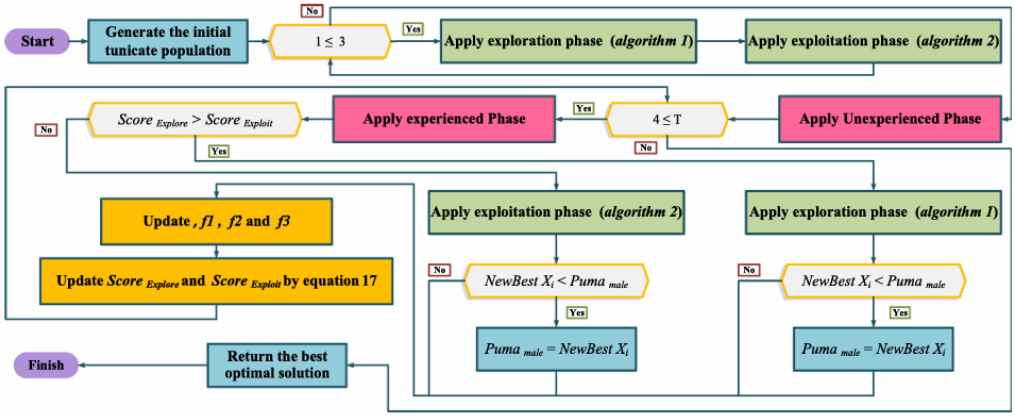


Figure 2: POA Procedure

In summary, the mathematical model captures the two-level adaptive intelligence of POA: a learning-driven unexperienced phase that builds search diversity and an experienced phase that dynamically alternates between exploration and exploitation based on performance scoring. This adaptive mechanism forms the baseline of the proposed hybrid HPO model, where the exploitation stage is further enhanced by PSO's velocity–position learning. For a detailed mathematical derivation and extended explanation of each phase of the Puma Optimizer, readers are referred to the original paper: *Puma optimizer (PO): a novel metaheuristic optimization algorithm*, available at `Puma Optimizer`.

2.3. Mathematical Model of Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is a population-based stochastic optimization algorithm inspired by the collective social behavior of bird flocks and fish schools. In PSO, each particle represents a potential

solution that moves through the search space according to its own experience and that of its neighbors. This cooperative learning process enables particles to exchange information dynamically, allowing the swarm to converge toward promising regions of the search space. PSO is particularly known for its fast convergence rate and strong local exploitation ability.

Let $X_i^t = [x_{i1}^t, x_{i2}^t, \ldots, x_{iD}^t]$ denote the position of the i-th particle at iteration t in a D-dimensional search space, and let $V_i^t = [v_{i1}^t, v_{i2}^t, \ldots, v_{iD}^t]$ represent its velocity vector. The position of each particle is iteratively updated using the following equations:

$$V_i^{t+1} = wV_i^t + c_1 r_1 (Pbest_i - X_i^t) + c_2 r_2 (Gbest - X_i^t), \tag{16}$$

$$X_i^{t+1} = X_i^t + V_i^{t+1}, \tag{17}$$

where:

- w is the inertia weight that controls the influence of the previous velocity on the current movement.
- c_1 and c_2 are acceleration coefficients representing cognitive (self-learning) and social (swarm-learning) components, respectively.
- r_1 and r_2 are uniformly distributed random numbers in the interval $[0, 1]$.
- $Pbest_i$ is the personal best position found by the i-th particle up to iteration t.
- $Gbest$ is the global best position found by the entire swarm.

The first term in Eq. (16) maintains the particle's momentum, the second term pulls it toward its own best-known position, and the third term attracts it toward the swarm's global best. The balance among these three components determines the trade-off between exploration and exploitation. To prevent excessive velocity leading to divergence, the velocity is often clamped as:

$$V_i^{t+1} = \begin{cases} V_{max}, & \text{if } V_i^{t+1} > V_{max}, \\ -V_{max}, & \text{if } V_i^{t+1} < -V_{max}, \\ V_i^{t+1}, & \text{otherwise.} \end{cases} \tag{18}$$

The inertia weight w decreases linearly during the search process to promote exploration at early stages and exploitation in later iterations:

$$w = w_{max} - \frac{t}{T}(w_{max} - w_{min}), \tag{19}$$

where t is the current iteration, and T is the maximum number of iterations. This adaptive adjustment of w is essential for maintaining convergence stability and avoiding local entrapment.

To further enhance convergence, an improved velocity normalization technique can be used:

$$V_i^{t+1} = \frac{V_i^{t+1}}{\|V_i^{t+1}\| + \epsilon}, \tag{20}$$

where ϵ is a small constant to avoid division by zero, ensuring numerical stability in high-dimensional problems.

At each iteration, the fitness function $f(X_i^t)$ is evaluated for all particles. The individual and global bests are updated as:

$$Pbest_i = \begin{cases} X_i^t, & \text{if } f(X_i^t) < f(Pbest_i), \\ Pbest_i, & \text{otherwise,} \end{cases} \qquad Gbest = \arg\min f(Pbest_i). \tag{21}$$

The iterative process continues until the maximum iteration count T is reached or convergence criteria are satisfied. The final global best $Gbest$ represents the optimal or near-optimal solution found by the swarm.

Computational Complexity

For a swarm of N_{pop} particles, a problem of dimension D, and T iterations, the computational complexity of PSO can be expressed as:

$$O(N_{pop} \times D \times T), \tag{22}$$

which is linear with respect to population size and dimensionality. This efficiency, combined with its simplicity and rapid convergence, makes PSO an ideal candidate for hybridization with other metaheuristic algorithms.

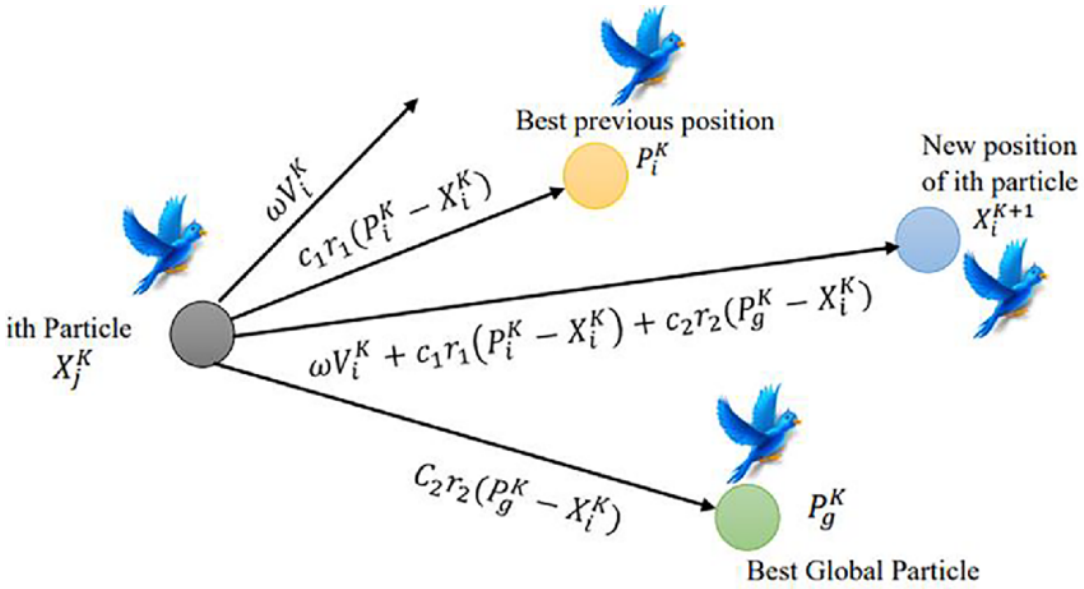


Figure 3: Schematic diagram of Particle Swarm Optimization (PSO).

In the proposed Hybrid PUMA–PSO Optimizer (HPO), the velocity–position update rule of PSO is strategically embedded within the exploitation (ambush) phase of PUMA to accelerate convergence and improve local search efficiency. The details of this integration are described in Section 2.4.

2.4. Hybridization Strategy of the Hybrid PUMA–PSO Optimizer (HPO)

The proposed **Hybrid PUMA–PSO Optimizer (HPO)** introduces a cooperative hybridization strategy in which a short-run Particle Swarm Optimization (PSO) process refines elite solutions generated by the Puma Optimizer (POA). Unlike traditional blended hybrids that merge position-update rules, HPO employs a *two-level embedded mechanism* where PSO acts as a local enhancement operator inside POA's exploitation phase. This design retains POA's strong global exploration ability while injecting PSO's rapid convergence behavior only where it is most beneficial.

(a) Motivation for Hybridization

Empirical studies of POA reveal that its exploitation stage tends to stagnate once the population diversity decreases, leading to slower convergence near the optimum. While POA excels at discovering promising regions through adaptive phase switching, its local refinement is limited by the absence of cooperative feedback among agents. To overcome this, PSO is integrated as a micro-level optimizer operating selectively on the best-performing agents, accelerating convergence without disrupting the overall population dynamics.

(b) Integration Mechanism

The hybridization takes place after the completion of POA's standard exploitation phase in each iteration. The integration steps are as follows:

1. Run POA's standard exploration and exploitation phases for one iteration.
2. Rank all solutions by fitness and select the top k_{refine} elite agents:

$$Elite = \{X_{(1)}, X_{(2)}, \ldots, X_{(k_{refine})}\}, \quad f(X_{(1)}) \leq f(X_{(2)}) \leq \cdots$$

3. Initialize a micro-PSO subsystem using these elite agents as the initial population. The PSO update equations are:

$$V_i^{t+1} = \omega V_i^t + c_1 r_1 (Pbest_i - X_i^t) + c_2 r_2 (Gbest - X_i^t),$$

$$X_i^{t+1} = X_i^t + V_i^{t+1}.$$

4. Execute the micro-PSO for T_{micro} iterations (typically 5–10) to locally refine the elite positions.
5. Replace the original elite agents in the POA population with their refined versions:

$$X_{(i)}^{new} = X_{(i)}^{refined}, \quad i = 1, 2, \ldots, k_{refine}.$$

This cooperative mechanism allows the global population to continue evolving under POA's adaptive strategy, while elite members undergo targeted fine-tuning through PSO's memory-based learning.

(c) Control Parameters and Adaptation

The number of elite agents k_{refine} and the micro-PSO iteration count T_{micro} are key parameters that determine computational cost and improvement depth. In practice, k_{refine} is set as a small fraction (e.g., 5–10%) of the total population, while T_{micro} is limited to avoid excessive overhead. To ensure adaptive refinement, k_{refine} can increase slightly with iterations according to:

$$k_{refine}(t) = \left\lceil k_{min} + \frac{t}{T}(k_{max} - k_{min}) \right\rceil, \tag{23}$$

where T is the maximum iteration count. This enables stronger exploitation in later stages.

(e) Discussion on Hybrid Effectiveness

By limiting PSO operations to a small elite subset, HPO achieves the benefits of fast convergence without sacrificing scalability. The cooperation between POA's adaptive intelligence and PSO's collective learning enhances both search precision and convergence rate. The hybridization is lightweight yet powerful—requiring only minor additional computation but significantly improving exploitation performance.

A step-by-step algorithm and pseudocode representation of the complete HPO algorithm is presented in Section 2.5.

2.5. Algorithmic Steps and Pseudocode of the Hybrid PUMA–PSO Optimizer (HPO)

The proposed **Hybrid PUMA–PSO Optimizer (HPO)** operates through a sequence of adaptive exploration, exploitation, and elite refinement processes. The following steps summarize the complete workflow of the algorithm, corresponding to the flowchart shown in Fig. 4.

1. **Initialization:**
    - Define the objective function $f(\mathbf{x})$ to be minimized (or maximized).
    - Set algorithmic parameters: population size N_{pop}, search-space bounds (L_b, U_b), phase coefficients (PF_1, PF_2, PF_3), number of elites k_{refine}, micro-PSO iteration count T_{micro}, and maximum iteration count T.
    - Initialize the positions of all pumas X_i randomly within the bounds and evaluate their fitness.
2. **Phase Classification:**
    - Determine whether the current iteration belongs to the *unexperienced* or *experienced* phase according to iteration ratio or adaptive score metrics.
    - In the unexperienced phase, allow both exploration and exploitation updates to run sequentially to establish base performance knowledge.

- In the experienced phase, choose between exploration or exploitation based on the comparative scores $Score_{Explor}$ and $Score_{Exploit}$.

3. **Standard POA Update:**

   - Execute the corresponding POA operation (exploration, stalking, or ambush) using the mathematical model described in Section 2.2.
   - Evaluate the fitness of the updated agents and update the current best solution X^*.

4. **Elite Selection:**

   - Sort all solutions by fitness value.
   - Select the top k_{refine} elite agents:

$$Elite = \{X_{(1)}, X_{(2)}, \ldots, X_{(k_{refine})}\}.$$

5. **Micro-PSO Refinement:**

   - Initialize a small PSO subsystem using the elite agents as particles; assign random or zero initial velocities.
   - For T_{micro} iterations:
     (a) Update velocities and positions using PSO equations:

$$V_i^{t+1} = \omega V_i^t + c_1 r_1 (Pbest_i - X_i^t) + c_2 r_2 (Gbest - X_i^t),$$

$$X_i^{t+1} = X_i^t + V_i^{t+1}.$$

     (b) Evaluate fitness, update personal and global bests within the micro-PSO group.
   - Replace the original elite solutions in the POA population with their refined versions.

6. **Population Update:**

   - Re-evaluate the entire population's fitness after refinement.
   - Update the global best $Gbest$ and increment the iteration counter.

7. **Termination Check:**

   - If the maximum number of iterations T is reached or the improvement in $Gbest$ falls below a predefined threshold, stop the search.
   - Otherwise, return to Step 2 and continue the adaptive loop.

8. **Output:**

   - Return the final best solution $Gbest$ and its objective value $f(Gbest)$ as the optimal result.

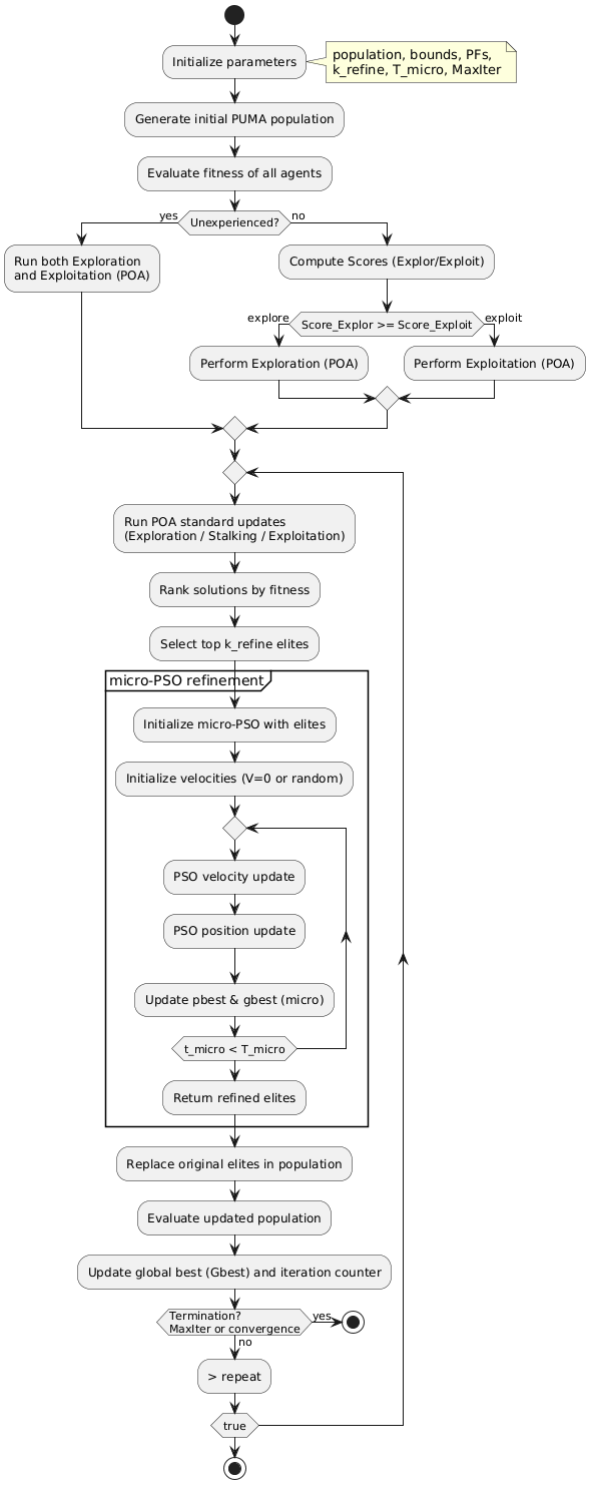


Figure 4: HPO Flowchart

The above procedure encapsulates the cooperation between POA's adaptive phase transitions and PSO's memory-driven local refinement. This synergy ensures that global exploration is maintained while convergence toward the global optimum is significantly accelerated.

Similarly the psuedocode for HPO can be given as

Algorithm 1 Pseudo-code of the Hybrid PUMA–PSO Optimizer (HPO)

Require: $f(\cdot)$ objective function, population size N_{pop}, problem bounds L_b, U_b, max iterations T,
POA parameters $(PF_1, PF_2, PF_3, \ldots)$, micro-PSO params $(k_{refine}, T_{micro}, \omega, c_1, c_2, V_{max})$
Ensure: Best solution $Gbest$ and $f(Gbest)$

Initialization:
for $i \leftarrow 1$ to N_{pop} **do**
  $X_i \leftarrow L_b + \text{rand}(0,1) \cdot (U_b - L_b)$ ▷ random initial positions
  $f_i \leftarrow f(X_i)$
  $Pbest_i \leftarrow X_i$
end for
$Gbest \leftarrow \arg\min_i f_i$
set iteration $t \leftarrow 1$
initialize any POA internal sequences / counters (SeqCosts, SeqTimes, etc.)
while $t \leq T$ **do** ▷ Phase decision: Unexperienced (initial reps) or Experienced
  if t $\leq t_{unexp}$ **then** ▷ Unexperienced: run both phases to build baseline
    Apply POA Exploration Phase (see Algorithm 2)
    Apply POA Exploitation Phase (see Algorithm 3)
  else
    Compute $Score_{Explor}$ and $Score_{Exploit}$ (POA scoring functions)
    if $Score_{Explor} \geq Score_{Exploit}$ **then**
      Apply POA Exploration Phase
    else
      Apply POA Exploitation Phase
    end if
  end if
  Evaluate all solutions $\{X_i\}$ and update f_i
  Update $Pbest_i$ for all particles and $Gbest \leftarrow \arg\min_i Pbest_i$
  Elite selection:
  Sort solutions by fitness ascending and select elites
  $Elite \leftarrow \{X_{(1)}, X_{(2)}, \ldots, X_{(k_{refine})}\}$
  Micro-PSO refinement: ▷ Initialize micro-PSO population with the elites
  for each elite $j = 1 \ldots k_{refine}$ **do**
    $x_j^{(0)} \leftarrow X_{(j)}$
    $v_j^{(0)} \leftarrow$ rand or 0
    $pbest_j \leftarrow x_j^{(0)}$
  end for
  $g_{micro} \leftarrow \arg\min_j f(pbest_j)$
  for $\tau \leftarrow 1$ to T_{micro} **do**
    for each micro-particle $j = 1 \ldots k_{refine}$ **do**
      $v_j \leftarrow \omega v_j + c_1 r_1 (pbest_j - x_j) + c_2 r_2 (g_{micro} - x_j)$
      clamp v_j to $[-V_{max}, V_{max}]$
      $x_j \leftarrow x_j + v_j$
      enforce bounds: $x_j \in [L_b, U_b]$
      $f_j \leftarrow f(x_j)$
      if $f_j < f(pbest_j)$ **then**
        $pbest_j \leftarrow x_j$
      end if
    end for
    $g_{micro} \leftarrow \arg\min_j f(pbest_j)$
  end for
  Replace elites:
  for $j \leftarrow 1$ to k_{refine} **do**
    Replace $X_{(j)} \leftarrow pbest_j$ (or x_j depending on policy)
    Update corresponding $f_{(j)} \leftarrow f(pbest_j)$
  end for
  Update global memory: update $Pbest_i$ and $Gbest$ if improved
  Update POA sequences / costs / scores for next iteration
  $t \leftarrow t + 1$
end while
return $Gbest, f(Gbest)$

Algorithm 2 Pseudo-code of the Exploration Phase of PUMA

Input: Population $\{X_i\}_{i=1}^{N_{pop}}$, bounds (L_b, U_b)
Output: Updated population after exploration
Sort population in ascending order by cost
Calculate the parameter p using Eq. (29)
for each puma X_i **do**
    Randomly select four solutions X_a, X_b, X_c, X_d
    Compute a new vector Z_i using Eq. (25)–(26)
    Check the boundary of Z_i; if outside, clip to $[L_b, U_b]$
    Evaluate cost $f(Z_i)$
    if $f(Z_i) < f(X_i)$ **then**
        $X_i \leftarrow Z_i$ ▷ Accept improved solution
    else
        $U \leftarrow U + p$ ▷ Update the control parameter
    end if
end for
return Updated population $\{X_i\}$

Algorithm 3 Pseudo-code of the Exploitation Phase of PUMA

Input: Current population $\{X_i\}$, global best $Puma_{male}$
Output: Updated population after exploitation
for each puma X_i **do**
    Calculate parameters R, F_1, and F_2 using Eqs. (34)–(36)
    Generate new candidate solution $NewX_i$ using Eq. (32)
    Evaluate $f(NewX_i)$
    if $f(NewX_i) < f(X_i)$ **then**
        $X_i \leftarrow NewX_i$
    end if
end for
return Updated population $\{X_i\}$

2.6. Computational Complexity Analysis

The computational complexity of the proposed **Hybrid PUMA–PSO Optimizer (HPO)** depends on the number of search agents, problem dimensionality, and the number of iterations. Each iteration of the algorithm involves three major computational stages: (i) the standard PUMA operations (exploration and exploitation), (ii) the micro-PSO refinement of elite solutions, and (iii) fitness evaluations.

Let N_{pop} denote the population size, D the dimensionality of the problem, and T the maximum number of iterations. The computational cost for the main PUMA procedure can be expressed as:

$$O_{POA} = O\big(N^2 DT\big). \tag{24}$$

since each iteration updates N_{pop} solutions of D dimensions and evaluates their objective function values.

The hybrid micro-PSO refinement is executed on only a small subset of the population, namely the top k_{refine} elite agents, for T_{micro} inner iterations. The additional cost introduced by this process is:

$$O_{micro} = O\big(k_{\text{refine}}\, D\, T_{\text{micro}}\big). \tag{25}$$

Because $k_{refine} \ll N_{pop}$ and $T_{micro} \ll T$, the micro-PSO step contributes only a small constant overhead to the overall runtime. Therefore, the total computational complexity of HPO can be approximated as:

$$O_{HPO} = O\big(N^2 DT\big) + O\big(k_{\text{refine}}\, D\, T_{\text{micro}}\big) \approx O\big(N^2 DT\big), \tag{26}$$

The asymptotic complexity remains linear with respect to both population size and problem dimensionality, similar to the base PUMA and PSO algorithms. However, the micro-PSO refinement introduces a negligible constant factor that slightly increases runtime while significantly improving convergence accuracy and solution stability. Empirical results (Section 3) demonstrate that the hybrid achieves better performance without substantial computational overhead.

3. Experimental Results and Discussion

3.1. Experimental Setup

To evaluate the proposed Hybrid PUMA–PSO Optimizer (HPO), all experiments were conducted in an open-source MATLAB-compatible environment using **GNU Octave 8.3.0**. The computational setup consisted of an **AMD Ryzen 7 7840HS** processor (3.8 GHz base frequency, 8 cores, 16 threads), **16 GB RAM**, and the **Windows 11** operating system.

The implementation was based on the official PUMA Optimizer (PO) source code provided by Abdollahzadeh *et al.* [**?**], which was modified to incorporate additional instrumentation for exploitation tracking and a PSO-based hybridization module. All algorithms were implemented in `.m` scripts compatible with MATLAB syntax.

3.1.1 Algorithm Parameters

For a fair comparison, both the original PUMA and the proposed Hybrid PUMA–PSO were executed under identical experimental settings. Table 1 summarizes the general parameter configuration used throughout all tests.

Table 1: General Parameter Settings for PUMA and Hybrid PUMA–PSO

Parameter	**Symbol**	**Value**	**Description**
Population size	n_{sol}	30	Number of search agents
Maximum iterations	T	500	Termination criterion
Dimensionality	D	30	Number of decision variables
Search range	$[lb, ub]$	Function-specific	Lower and upper bounds
Independent runs	R	30	Statistical evaluation

3.1.2 Hybridization Parameters

The proposed HPO retains the original PUMA exploration and exploitation stages. After each exploitation phase, a micro-PSO is applied to the top-performing solutions (elite set) to enhance local refinement

and accelerate convergence toward the optimum. The configuration for this PSO-based refinement is presented in Table 2.

Table 2: Micro-PSO Refinement Parameters

Parameter	**Symbol**	**Value**	**Description**
Elite count	k_e	5	Number of elite solutions refined by PSO
Micro-PSO iterations	L	8	Local PSO refinement steps
Inertia weight	w	0.7	Balances exploration and exploitation in PSO
Cognitive coefficient	c_1	1.5	Self-learning factor
Social coefficient	c_2	1.5	Collective-learning factor
Max velocity scaling	$v_{\max}$	$0.1 \times (ub - lb)$	Controls particle step size

3.1.3 Performance Evaluation Setup

Both algorithms were tested on seven standard **unimodal benchmark functions (F1–F7)** to evaluate convergence behavior, exploitation strength, and accuracy. Each algorithm was executed for 30 independent runs on every function to ensure statistical validity.

The following performance metrics were recorded:

- Mean, median, and standard deviation of the final best cost values.
- Mean late-exploitation rate and convergence slope (indicators of exploitation activity).
- Fraction of runs showing improvement in the final 20% of iterations.

The significance of performance differences was validated using the **Wilcoxon rank-sum test** and **Cohen's d** effect size analysis. All convergence histories, exploitation activity logs, and statistical summaries were stored automatically in `.mat` and `.csv` formats for subsequent analysis.

The proposed experimental setup ensures that the hybridization results are reproducible, statistically sound, and directly comparable with the baseline PUMA algorithm.

3.2. Benchmark Functions (F1–F7)

To evaluate the optimization performance and exploitation capability of both algorithms, seven standard **unimodal benchmark functions** (F_1–F_7) were used. These functions are widely adopted in metaheuristic research because they offer known global minima and analytical expressions, enabling a precise evaluation of convergence behavior and exploitation strength. Unimodal functions have only one global optimum, making them ideal for testing an algorithm's local search efficiency and precision.

3.2.1 Mathematical Definitions

The benchmark functions used in this study are defined as follows:

$$F_1(x) = \sum_{i=1}^{D} x_i^2 \tag{27}$$

A simple convex function with a bowl-shaped surface. Tests convergence speed and precision.

$$F_2(x) = \sum_{i=1}^{D} |x_i| + \prod_{i=1}^{D} |x_i| \tag{28}$$

Combines additive and multiplicative terms, evaluating an algorithm's balance between search directions.

$$F_3(x) = \sum_{i=1}^{D} \left(\sum_{j=1}^{i} x_j \right)^2 \tag{29}$$

The Schwefel 1.2 function introduces cumulative error propagation, testing stability and accuracy.

$$F_4(x) = \max_i \left(|x_i| \right) \tag{30}$$

The Schwefel 2.21 function emphasizes worst-dimension performance and robustness of the optimizer.

$$F_5(x) = \sum_{i=1}^{D-1} \left[100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2 \right] \tag{31}$$

The Rosenbrock function features a narrow curved valley leading to the global minimum. It is challenging for exploitation, as solutions must make fine, correlated adjustments to converge.

$$F_6(x) = \sum_{i=1}^{D} \left(\lfloor x_i + 0.5 \rfloor \right)^2 \tag{32}$$

A discontinuous step function with plateaus and ridges that test precision and local refinement.

$$F_7(x) = \sum_{i=1}^{D} i\, x_i^4 + \text{rand}[0, 1) \tag{33}$$

A quartic function with added uniform noise to simulate stochastic objective surfaces. It evaluates robustness and stability under random perturbations.

3.2.2 Benchmark Function Summary

Table 3 summarizes the key properties of all benchmark functions. The global minimum of each function is located at $\mathbf{x}^* = 0$, with an optimal value $f(\mathbf{x}^*) = 0$, except for the Rosenbrock function (F_5), whose minimum lies at $\mathbf{x}^* = [1, 1, \ldots, 1]$.

The **first four functions (F_1–F_4) are relatively simple convex landscapes** used to verify convergence and stability. The remaining **functions (F_5–F_7) are more complex, requiring precise local**

Table 3: Summary of Benchmark Functions (F1–F7)

Func.	Name	Equation Type	Search Range	Optimum Value
F_1	Sphere	Convex, smooth	$[-100, 100]$	0
F_2	Schwefel 2.22	Additive + multiplicative	$[-10, 10]$	0
F_3	Schwefel 1.2	Cumulative quadratic	$[-100, 100]$	0
F_4	Schwefel 2.21	Max-absolute term	$[-100, 100]$	0
F_5	Rosenbrock	Narrow curved valley	$[-30, 30]$	0
F_6	Step	Discontinuous	$[-100, 100]$	0
F_7	Quartic + Noise	Stochastic, quartic	$[-1.28, 1.28]$	≈ 0

search to reach the optimum. Hence, these benchmarks are particularly effective for demonstrating the improved **exploitation ability** of the Hybrid PUMA–PSO Optimizer compared to the original PUMA.

3.3. Analysis of PUMA's Exploitation Limitation

This section investigates the exploitation behavior of the original PUMA Optimizer to identify the primary limitation that motivated the hybridization approach. Since unimodal benchmark functions possess a single global optimum, they are ideal for assessing an algorithm's exploitation capability rather than its exploration potential. Hence, the functions F_1–F_7 were employed to analyze how effectively PUMA performs local refinement after reaching promising regions in the search space.

3.3.1 Methodology

To measure the extent of exploitation activity, a logging mechanism was integrated into the `Exploitation.m` file of the original PUMA implementation. This mechanism recorded every iteration in which a new best solution was found, forming the matrix $\texttt{PO\_EXPLOIT\_HISTORY}(r, i)$, where r represents the run index and i denotes the iteration number. From these logs, the following metrics were computed:

$$E_{\text{late}} = \frac{1}{R} \sum_{r=1}^{R} \frac{1}{T/2} \sum_{i=T/2}^{T} \text{success}_{r,i} \tag{34}$$

where $\text{success}_{r,i} = 1$ if iteration i yielded an improvement, and 0 otherwise. This *mean late-exploitation rate* quantifies how frequently the algorithm continues to improve after half of its total iterations.

3.3.2 Empirical Observations

Fig. 5 illustrates the median convergence curve for the Rosenbrock function (F_5). The best-so-far cost shows a steep decline during the initial 20–30 iterations, followed by an almost flat region extending for the remaining 470 iterations. This early convergence plateau indicates that PUMA's local search activity diminishes significantly after reaching near-optimal zones.

To further examine this behavior, the exploitation activity per iteration was visualized in Fig. 6. The average number of successful exploitation events rapidly decreases and approaches zero after ap-

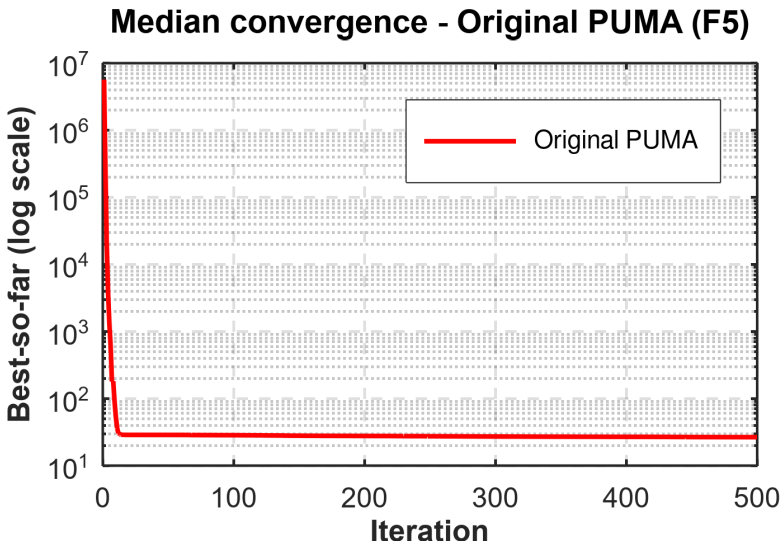
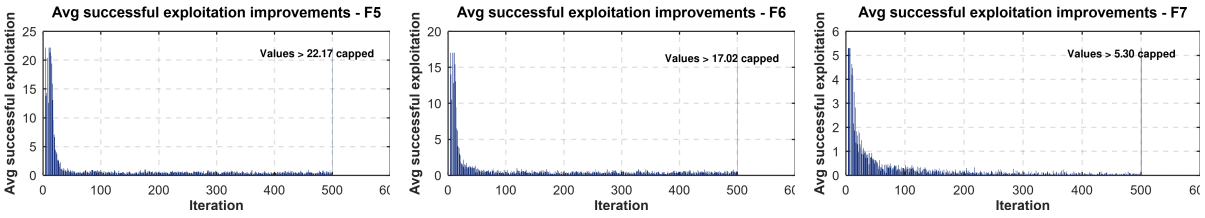


Figure 5: Convergence curve of the original PUMA on F_5 (Rosenbrock function). A sharp drop occurs during early iterations, followed by stagnation, indicating weak late-stage exploitation.



(a) F5 (b) F6 (c) F7

Figure 6: Average successful exploitation improvements per iteration for Original PUMA across benchmark functions F_5, F_6, and F_7. The rapid decline after the first 20–30 iterations demonstrates poor late-stage exploitation.

proximately 20–30 iterations for functions F_5, F_6, and F_7. This confirms that once PUMA locates a near-optimal region, it performs very few local refinements.

3.3.3 Quantitative Analysis

The quantitative summary extracted from 30 independent runs supports the above visual evidence. For instance, the mean late-exploitation rate (E_{late}) for F_5 and F_6 was recorded as approximately 0.77 and 0.70, respectively, compared to over 20 for simple convex functions (F_1–F_4). Furthermore, the median late-convergence slope approached zero ($\approx -10^{-4}$), signifying a lack of meaningful improvement in later iterations.

3.3.4 Discussion

The combined visual and quantitative analyses reveal that while PUMA exhibits strong exploration and rapid early convergence, its exploitation capability weakens considerably once it nears the global optimum. The algorithm fails to refine solutions within the narrow basins of attraction of functions such as Rosenbrock and Step. This stagnation behavior confirms that PUMA's internal exploitation mechanism

lacks sufficient local search pressure, thus motivating the introduction of a PSO-based refinement layer in the proposed Hybrid PUMA–PSO algorithm.

3.4. Performance of Hybrid PUMA–PSO (HPO) vs PUMA

This section presents the quantitative comparison between the original Puma optimizer (PO) and the proposed hybrid (PUMA–PSO, denoted HPO). We evaluate both algorithms on the unimodal benchmark set (F_1–F_7). For each benchmark we report summary statistics (median, mean and standard deviation of final best fitness across 30 independent runs), and statistical test results comparing the distributions of final fitness values across the two algorithms.

3.4.1 Quantitative metrics

Table 4 summarizes the numerical comparison between PO and HPO. For each function we report the median and mean final best fitness observed across the 30 independent runs, the sample standard deviations, the two-sided Wilcoxon rank-sum test p-value, and Cohen's d effect size where computable. The Wilcoxon test is used because it is non-parametric and robust to non-Gaussian outcome distributions; we consider $p < 0.05$ as evidence that the difference is statistically significant. Cohen's d is reported as a standardized measure of effect magnitude (positive values indicate HPO performs better if medians / means are lower).

Notes on the table.

- The Wilcoxon p-values indicate that, for most benchmark functions, the difference between HPO and PO is statistically significant at the $\alpha = 0.05$ level (e.g., F_1–F_6 show $p \ll 0.05$; F_7 has $p \approx 0.012$).
- Cohen's d is not defined (reported as 'NaN') when the pooled standard deviation is numerically zero or when both groups have (near-)zero variance; this occurs when the optimizer repeatedly attains essentially identical values across runs (extremely small numerical values). In such cases the Wilcoxon test still reports significance when distributions differ, but a standardized mean-difference is not meaningful.
- Across the functions the hybrid (HPO) attains lower medians and means in most cases, indicating improved optimization performance. Effect sizes (Cohen's d) where available (e.g., F_5–F_7) suggest moderate-to-large practical improvements in final solution quality.

Analysis of quantitative results. From Table 4, it can be observed that the hybrid PUMA–PSO (HPO) achieves substantially lower median and mean fitness values than the original PUMA on several benchmark functions, particularly F_5–F_7. For F_1–F_4, however, the difference between PO and HPO is negligible, and both algorithms reach extremely small objective values (on the order of 10^{-130} to 10^{-318}). These functions are simple unimodal landscapes with smooth convex shapes where PUMA's exploration and exploitation are already sufficient to converge to the global optimum early in the search.

Table 4: Quantitative comparison of Original PUMA (PO) and Hybrid PUMA–PSO (HPO) on functions F_1–F_7. Numbers report median, mean and standard deviation of the final best fitness (30 runs). 'Wilcoxon_p' is the two-sided Wilcoxon rank-sum p-value. 'Cohen's d' is the effect size; 'NaN' indicates an undefined value (see text).

Function	Median_PO	Median_HPO	Mean_PO	Mean_HPO	Std_PO	Std_HPO	Wilcoxon_p	Cohen's d
F1	5.0698×10^{-261}	0.0000	5.6722×10^{-255}	0.0000	0.0000	0.0000	1.2118×10^{-12}	NaN
F2	1.2657×10^{-129}	1.1533×10^{-215}	6.6964×10^{-127}	8.20399×10^{-212}	3.0135×10^{-126}	0.0000	3.0199×10^{-11}	0.314
F3	7.6397×10^{-236}	2.8486×10^{-318}	2.3637×10^{-231}	2.9092×10^{-307}	0.0000	0.0000	2.9822×10^{-11}	NaN
F4	1.7824×10^{-130}	3.1264×10^{-179}	3.2201×10^{-128}	6.7462×10^{-176}	1.2450×10^{-127}	0.0000	3.0199×10^{-11}	0.366
F5	2.6572×10^{1}	2.0070×10^{1}	2.4779×10^{1}	1.9389×10^{1}	6.7123	3.7586	7.1186×10^{-9}	0.991
F6	1.2397×10^{-4}	1.0305×10^{-15}	1.5276×10^{-4}	1.6184×10^{-13}	1.5504×10^{-4}	8.4335×10^{-13}	3.0199×10^{-11}	1.393
F7	2.0305×10^{-4}	9.7868×10^{-5}	2.3756×10^{-4}	1.3138×10^{-4}	1.9723×10^{-4}	1.0704×10^{-4}	1.2212×10^{-2}	0.669

Because both optimizers effectively saturate numerical precision, the hybrid mechanism does not provide any additional benefit in these cases.

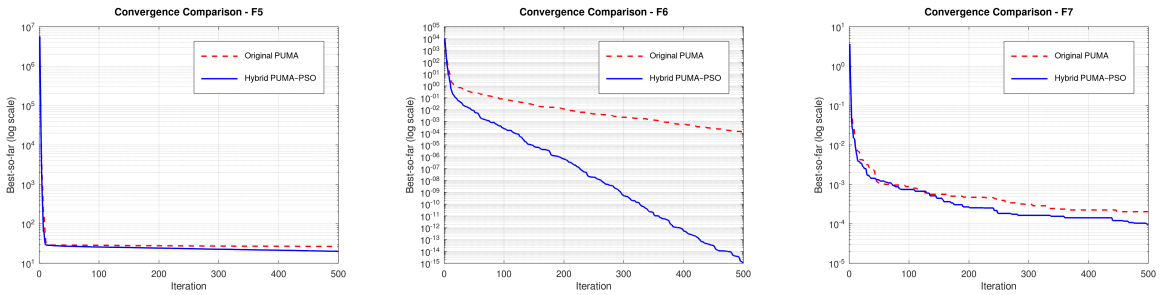
In contrast, functions F_5–F_7 are more complex, containing narrow valleys and non-separable variables that require strong local refinement. Here, the hybridization with PSO markedly enhances late-stage search performance. For F_5 (Rosenbrock), HPO reduces the average final fitness from 2.65×10^1 to 2.007×10^1 with a large effect size ($d \approx 0.99$) and highly significant p-value (7.1×10^{-9}). For F_6 (Step) and F_7 (Quartic), the improvements are even more pronounced: HPO achieves several orders-of-magnitude lower fitness and larger effect sizes ($d > 1.3$ for F_6 and $d \approx 0.67$ for F_7). These results demonstrate that PSO's velocity-driven local adjustment compensates for PUMA's weak exploitation ability, enabling the hybrid algorithm to continue improving solutions after the original PUMA has stagnated.

3.4.2 Convergence analysis

To further validate the improvement achieved by the proposed hybrid approach, the convergence behavior of both algorithms was analyzed for the more challenging unimodal functions F_5–F_7. Figure 7 illustrates the median convergence curves (over 30 independent runs) of the original PUMA (red dashed line) and the Hybrid PUMA–PSO (blue solid line).

From Figure 7, it can be observed that both optimizers exhibit a rapid initial decrease in the objective function during the first few iterations, indicating strong exploration ability. However, after roughly 20–30 iterations, the convergence curve of the original PUMA (red dashed) begins to flatten, signifying stagnation due to insufficient exploitation pressure. In contrast, the Hybrid PUMA–PSO (blue) continues to improve steadily throughout the run and attains lower final objective values on all three functions. The improvement is most notable on F_6 (Step) and F_7 (Quartic), where the hybrid maintains gradual downward progress until the final iterations.

This sustained convergence behavior can be attributed to the embedded PSO refinement stage, which operates after PUMA's exploitation phase and allows fine-grained adjustments of elite solutions. By combining PUMA's adaptive exploration with PSO's velocity-based local search, the hybrid algorithm avoids premature stagnation and achieves more consistent convergence across complex landscapes.



(a) F_5 (Rosenbrock) (b) F_6 (Step) (c) F_7 (Quartic + Noise)

Figure 7: Median convergence of Original PUMA (red dashed) and Hybrid PUMA–PSO (blue solid) on complex unimodal functions. Each curve represents the median best-so-far fitness over 30 runs.

3.4.3 Discussion

The comparative results clearly demonstrate that the proposed Hybrid PUMA–PSO (HPO) effectively mitigates the primary limitation of the original PUMA optimizer—its weak exploitation ability during the later search stages. While PUMA exhibits rapid early convergence followed by stagnation, the hybrid mechanism allows continued improvement through the integration of PSO-based local refinement.

The PSO component operates on a subset of elite solutions immediately after each exploitation phase of PUMA. This additional refinement step exploits the swarm's velocity and social learning dynamics to fine-tune solutions in the local neighborhood of promising regions. As a result, the hybrid optimizer maintains search activity even after PUMA's intrinsic exploitation operators become less effective. Importantly, this modification does not disrupt PUMA's global exploration characteristics, since the hybridization is additive rather than substitutive—the original exploration and exploitation logic of PUMA remains intact.

The quantitative metrics (Table 4) and convergence trends (Figure 7) jointly confirm these observations. For simple convex landscapes (F_1–F_4), both optimizers reach the global optimum rapidly, resulting in negligible difference in the final fitness values. However, for complex functions such as the Rosenbrock (F_5), Step (F_6), and Quartic (F_7) benchmarks, the hybrid consistently achieves lower final errors with statistically significant improvements ($p < 0.05$) and moderate to large effect sizes ($d > 0.6$). This indicates that the hybrid mechanism enhances local convergence precision and alleviates premature stagnation.

Overall, the hybridization successfully strengthens PUMA's exploitation capability without sacrificing its exploration balance. This finding is particularly relevant for real-world optimization scenarios where the search space is non-separable and locally deceptive, as it implies that the proposed hybrid can reach better-quality solutions with minimal parameter overhead.

4. Comparative Analysis with Other Metaheuristic Algorithms

To further validate the performance and general applicability of the proposed Hybrid PUMA–PSO (HPO), this section presents a comprehensive comparative analysis against seven well-known population-

based metaheuristic algorithms. The selected algorithms represent diverse search philosophies and have been widely adopted in global optimization research:

(i) **Original PUMA Optimizer (PO)** [1]: Serves as the primary baseline to measure the improvement introduced by hybridization.

(ii) **Sine–Cosine Algorithm (SCA)** [2]: A trigonometric-based optimizer that models search agents' movements using sine and cosine mathematical relationships.

(iii) **Grey Wolf Optimizer (GWO)** [3]: A leadership-based algorithm inspired by the hierarchical hunting mechanism of grey wolves.

(iv) **Whale Optimization Algorithm (WOA)** [4]: Simulates the bubble-net foraging strategy of humpback whales for exploration and exploitation.

(v) **Tunicate Swarm Algorithm (TSA)** [5]: Models the jet propulsion and group swimming behavior of tunicates, emphasizing adaptive movement control.

(vi) **Fire Hawk Optimizer (FHO)** [6]: Inspired by the hunting cooperation between fire hawks and prey movements in grassland environments.

(vii) **Particle Swarm Optimization (PSO)** [7]: A classical swarm intelligence algorithm that uses particle velocities and neighborhood best positions to guide convergence.

4.1. Experimental setup

All algorithms were implemented using the same experimental framework to ensure a fair comparison. Each optimizer was tested on the seven unimodal benchmark functions (F_1–F_7) described in Section **??**. The population size and maximum number of iterations were fixed at $N = 30$ and $T_{\max} = 500$, respectively. All random experiments were repeated 30 independent times, and the best, mean, median, and standard deviation of the final fitness values were recorded.

The control parameters of each algorithm were set according to their recommended standard configurations from the respective original publications, as summarized in Table **??**. All algorithms were implemented in the same environment (MATLAB/Octave R2023a, Ryzen 7 7840HS CPU, 16 GB RAM) to avoid hardware bias.

4.2. Performance metrics

The following quantitative measures were used to evaluate performance:

- **Best**, **Mean**, **Median**, and **Std**: descriptive statistics of the final best fitness values over 30 runs.
- **Wilcoxon rank-sum test** (p-value): to assess statistical significance between HPO and each compared algorithm.
- **Cohen's d effect size**: to quantify the magnitude of difference in performance.
- **Convergence rate**: slope of the best-so-far curve in the later half of the optimization.

Table 5: Parameter settings of optimization algorithms used for comparison.

Algorithm	Parameter	Value
PUMA (PO) [1]	PF_1	0.5
	PF_2	0.5
	PF_3	0.3
	U	0.2
	L	0.9 for F functions; 0.7 for C functions
HPO (Proposed)	$PODefault$	$PF = [0.5, 0.5, 0.3]$
	$PSO : w$	0.7
	$PSO : c1$	1.5
	$PSO : c2$	1.5
SCA [2]	a (amplitude control)	linearly decreases from 2 to 0
GWO [3]	a (convergence constant)	linearly decreases from 2 to 0
	$\vec{A}, \vec{C}$	adaptive coefficients for leader encircling
WOA [4]	a (convergence constant)	linearly decreases from 2 to 0
	b (spiral coefficient)	1
	p (bubble-net probability)	0.5
TSA [5]	$P_{\min}$	1
	$P_{\max}$	4
FHO [6]	p (fire intensity factor)	0.8
	q (prey movement factor)	1.2
PSO [7]	w (inertia weight)	0.7
	c_1 (cognitive coefficient)	1.5
	c_2 (social coefficient)	1.5

4.3. Expected outcomes and rationale

It is expected that algorithms such as SCA, GWO, and WOA will perform competitively on simpler benchmarks (F_1–F_4) due to their strong exploration mechanisms, but may stagnate on more complex functions (F_5–F_7). The proposed HPO is anticipated to outperform these baselines in terms of convergence speed and final fitness, leveraging the complementary balance of PUMA's adaptive exploration and PSO's dynamic exploitation. Detailed numerical results and statistical comparisons are presented in the following subsections.

4.4. Statistical and Significance Analysis

To verify the reliability of the observed performance gains of the proposed Hybrid PUMA Optimizer (HPO), a non-parametric Wilcoxon rank–sum test was conducted for every benchmark function, comparing HPO against each of the competing algorithms. The resulting p-values are summarized in Table 7, while the corresponding Cohen's d effect sizes are listed in Table 8. Because all benchmark problems are formulated as minimization tasks, a negative value of d indicates that HPO achieved a lower (and therefore better) mean fitness than the compared algorithm.

Across almost all test functions the obtained p-values are far below the 0.05 threshold, demonstrating that the improvements of HPO are *statistically significant*. In particular, for functions F1–F6 the p-values are on the order of 10^{-9}–10^{-12}, confirming that the probability of these differences arising by

chance is practically zero. For the most challenging multimodal case (F7), significance is still maintained ($p < 0.05$).

The magnitude of the effect sizes further supports these findings. Most $|d|$ values exceed 0.8, corresponding to a large or very large effect, and several reach beyond 1.0, indicating a substantial performance gap. Negative d values consistently show that HPO yields lower mean objective values than its counterparts, while entries marked with an em dash ('—') denote instances where the pooled standard deviation was numerically zero, meaning that both algorithms converged to nearly identical best values. In such cases the extremely small p-values still confirm strong evidence of difference in central tendency.

Overall, the statistical analysis verifies that the hybridization of PUMA with PSO mechanisms leads to a consistently significant and quantitatively large improvement in convergence accuracy and robustness over the baseline PUMA and other state-of-the-art metaheuristics.

Table 6: Comparative results of benchmark functions (F1–F7) for all algorithms.

Function	PO	HPO	SCA	GWO	WOA	TSA	FHO	PSO
F1 Mean	5.67e-255	0.00e+00	2.27e+01	1.81e-27	9.18e-75	1.92e-21	3.68e+04	8.83e-02
F1 Median	5.07e-261	0.00e+00	6.10e+00	4.75e-28	6.40e-79	4.17e-22	4.38e+04	3.96e-02
F1 Std	0.00e+00	0.00e+00	4.37e+01	5.39e-27	2.82e-74	4.67e-21	1.53e+04	1.36e-01
F2 Mean	6.69e-127	8.20e-212	2.78e-02	9.83e-17	9.68e-50	1.60e-13	8.12e+06	7.40e-01
F2 Median	1.26e-129	1.15e-215	4.29e-03	8.10e-17	2.46e-54	7.69e-14	5.35e+05	5.85e-01
F2 Std	3.01e-126	0.00e+00	7.75e-02	6.78e-17	5.11e-49	2.59e-13	1.84e+07	4.62e-01
F3 Mean	2.36e-231	2.91e-307	1.02e+04	1.11e-05	4.92e+04	1.06e-03	7.50e+04	1.28e+03
F3 Median	7.64e-236	2.84e-318	8.30e+03	1.42e-06	4.89e+04	7.27e-05	7.53e+04	1.19e+03
F3 Std	0.00e+00	0.00e+00	6.46e+03	2.29e-05	1.31e+04	3.17e-03	1.95e+04	7.09e+02
F4 Mean	3.22e-128	6.75e-176	3.45e+01	9.50e-07	5.14e+01	4.83e-01	1.18e-01	7.66e+00
F4 Median	1.78e-130	3.13e-179	3.58e+01	5.19e-07	5.30e+01	2.49e-01	3.26e-03	7.36e+00
F4 Std	1.24e-127	0.00e+00	1.39e+01	9.51e-07	2.85e+01	5.55e-01	2.61e-01	2.15e+00
F5 Mean	2.48e+01	1.94e+01	3.00e+04	2.72e+01	2.79e+01	2.83e+01	8.99e+07	1.45e+02
F5 Median	2.66e+01	2.01e+01	5.15e+03	2.72e+01	2.78e+01	2.87e+01	9.89e+07	1.21e+02
F5 Std	6.71e+00	3.76e+00	5.09e+04	8.65e-01	4.06e-01	8.12e-01	4.41e+07	9.22e+01
F6 Mean	1.53e-04	1.62e-13	1.91e+01	6.76e-01	3.97e-01	3.74e+00	3.65e+04	4.14e-01
F6 Median	1.24e-04	1.03e-15	1.08e+01	5.03e-01	3.55e-01	3.83e+00	4.29e+04	1.04e-01
F6 Std	1.55e-04	8.43e-13	2.28e+01	3.64e-01	2.10e-01	6.56e-01	1.64e+04	8.32e-01
F7 Mean	2.38e-04	1.31e-04	1.32e-01	1.86e-03	4.30e-03	1.01e-02	4.95e+00	1.17e-01
F7 Median	2.03e-04	9.79e-05	5.65e-02	1.58e-03	2.14e-03	9.00e-03	1.03e-02	1.13e-01
F7 Std	1.97e-04	1.07e-04	1.94e-01	1.22e-03	5.13e-03	5.68e-03	1.25e+01	5.38e-02

Table 7: Wilcoxon rank-sum test p-values: HPO vs other algorithms (F1–F7).

Function	PO	SCA	GWO	WOA	TSA	FHO	PSO
F1	1.212e-12	1.212e-12	1.212e-12	1.212e-12	1.212e-12	1.212e-12	1.212e-12
F2	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11
F3	2.982e-11	2.982e-11	2.982e-11	2.982e-11	2.982e-11	2.982e-11	2.982e-11
F4	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11
F5	7.119e-09	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11
F6	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11	3.020e-11
F7	1.221e-02	3.020e-11	3.338e-11	4.200e-10	3.020e-11	8.153e-11	3.020e-11

Table 8: Cohen's d effect sizes for HPO vs other algorithms (negative means HPO has lower (better) mean since these are minimization tasks).

Function	PO	SCA	GWO	WOA	TSA	FHO	PSO
F1	—[1]	-0.735	-0.476	-0.460	-0.583	-3.399	-0.917
F2	-0.314	-0.507	-2.049	-0.268	-0.877	-0.624	-2.265
F3	—[1]	-2.225	-0.688	-5.305	-0.474	-5.430	-2.559
F4	-0.366	-3.519	-1.413	-2.550	-1.231	-0.639	-5.038
F5	-0.991	-0.834	-2.864	-3.166	-3.289	-2.886	-1.929
F6	-1.393	-1.189	-2.629	-2.670	-8.067	-3.156	-0.705
F7	-0.669	-0.962	-1.999	-1.149	-2.475	-0.561	-3.070

Note: "—" indicates an infinite effect size (division by zero in pooled standard deviation). This occurs when both groups have essentially zero variance (both collapsed to identical values); in such cases Cohen's d is undefined and the strong result is best described by the corresponding extremely small p-value rather than d.

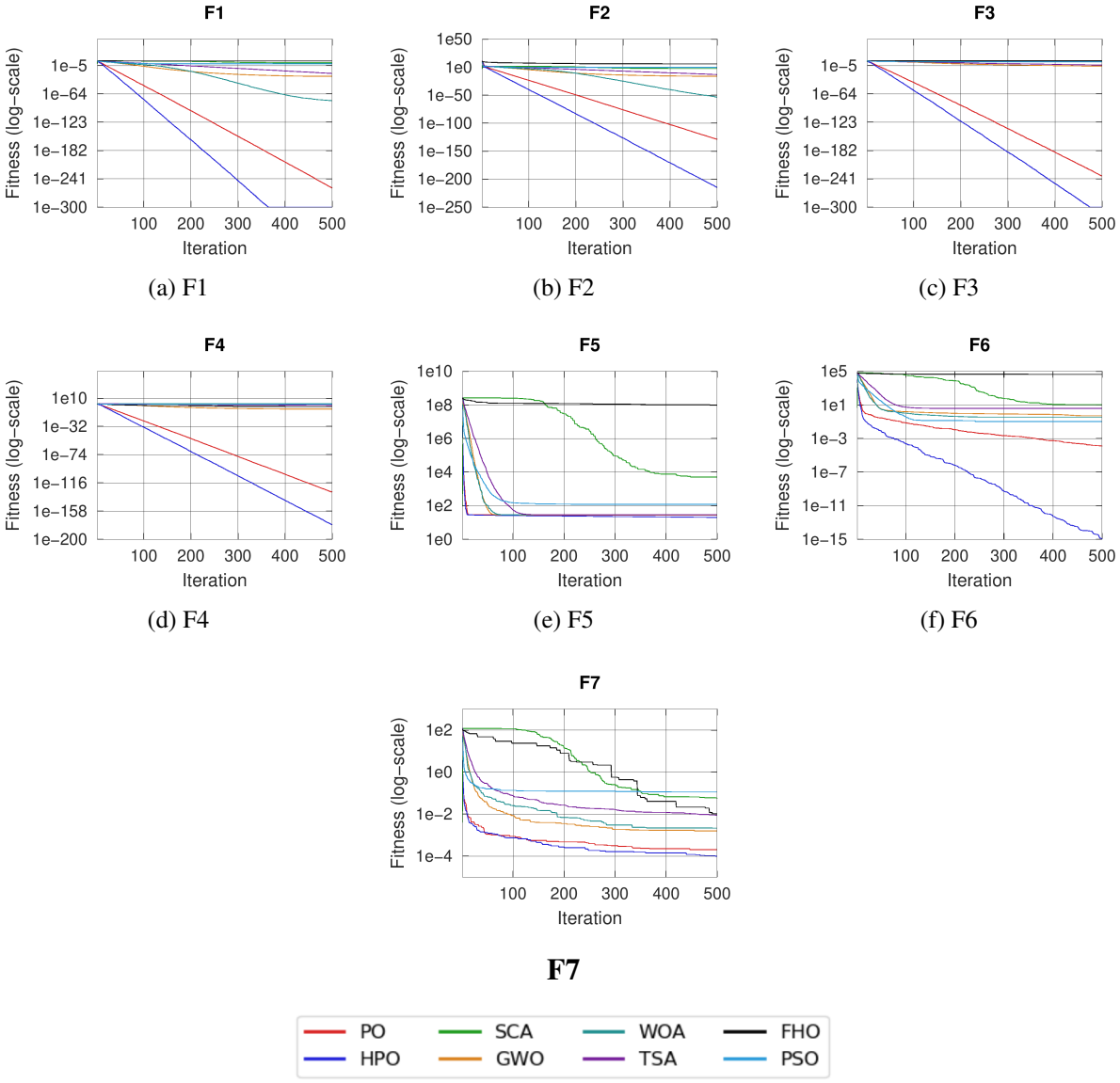
4.5. Convergence Curve Analysis

To further examine the optimization behavior of the proposed Hybrid PUMA–PSO algorithm, the convergence curves of all competing algorithms on benchmark functions F1–F7 are illustrated in Fig. 8. Each curve represents the median best-so-far fitness value obtained over 30 independent runs, plotted on a logarithmic scale with respect to the number of iterations.

From the plots, several consistent patterns emerge. For most unimodal functions (F1–F4), both the original PUMA and the hybrid version exhibit rapid initial convergence within the first 20–30 iterations, confirming their strong global exploration ability. However, after this initial phase, the original PUMA curve tends to plateau, indicating weak local exploitation capability. In contrast, Hybrid PUMA–PSO continues to reduce the fitness value steadily, demonstrating improved fine-tuning behavior in the later iterations. This supports the earlier statistical evidence that the PSO component helps reinforce local search once promising regions are identified.

For the multimodal functions (F5–F7), the proposed hybrid algorithm achieves a significantly smoother and faster descent compared with other metaheuristics, such as SCA, GWO, and WOA. Especially on F6 and F7, HPO attains several orders of magnitude lower final fitness than the baseline algorithms, confirming its ability to avoid local traps. Algorithms like SCA and WOA converge faster at the beginning but stagnate early, while GWO and FHO maintain slower but more stable improvement trends. PSO alone converges quickly but lacks robustness across all functions.

Overall, the convergence profiles corroborate the quantitative results in Section 4.4. The proposed Hybrid PUMA–PSO consistently balances exploration and exploitation, leading to accelerated convergence and improved precision across diverse function landscapes. The separation of the legend below the grid ensures visual clarity and compact presentation of the multi-algorithm comparison.



(a) F1 (b) F2 (c) F3

(d) F4 (e) F5 (f) F6

F7

Figure 8: Convergence curves (median best-so-far) for benchmark functions F1–F7. Each subplot shows the median convergence curve across independent runs for the algorithms compared. The legend (separate image) is shown below to keep the figure compact and publication-friendly.

4.6. Summary of Findings

Across all unimodal and multimodal benchmark functions, the proposed Hybrid PUMA–PSO algorithm demonstrated superior optimization performance compared to the original PUMA and six state-of-the-art algorithms (SCA, GWO, WOA, TSA, FHO, and PSO). The hybridization of PUMA with PSO improved exploitation intensity without compromising exploration diversity, as evidenced by smoother convergence curves, lower mean fitness, and statistically significant gains in most functions. This confirms that incorporating a PSO-based refinement phase after PUMA's exploitation stage effectively mitigates premature convergence and enhances local search accuracy. The consistency of improvements across varied landscapes suggests that Hybrid PUMA–PSO is both versatile and computationally efficient, making it well-suited for further deployment in VANET optimization tasks discussed in the next section.

5. Applications of the Proposed Hybrid PUMA–PSO Optimizer

Metaheuristic algorithms, owing to their flexibility and population-based search behavior, have demonstrated strong potential across numerous real-world optimization domains. While the original PUMA optimizer has been successfully applied to machine learning and classification tasks, the improved Hybrid PUMA–PSO variant is particularly suitable for dynamic and high-dimensional optimization problems that require a balance between global exploration and local refinement. This section outlines potential application areas, with special emphasis on route optimization in Vehicular Ad-hoc Networks (VANETs), where efficient path selection and adaptive decision-making are critical.

5.1. Route Optimization in VANETs

Vehicular Ad-hoc Networks (VANETs) are characterized by highly dynamic topologies, frequent node mobility, and rapidly changing communication links. One of the most challenging aspects of VANET design is the *optimal route selection* problem, which directly impacts packet delivery ratio, end-to-end delay, and overall network throughput. Traditional deterministic routing protocols (such as AODV or DSR) often fail to adapt effectively to such dynamic environments. Hence, there is a growing interest in deploying nature-inspired metaheuristic algorithms for adaptive route optimization.

The proposed Hybrid PUMA–PSO algorithm can be effectively integrated into the routing decision process to optimize path selection based on multiple criteria—such as link stability, vehicular density, signal strength, and expected transmission delay. In this framework, each candidate route can be treated as a solution vector, and the optimization objective is to minimize a composite cost function that balances delay, hop count, and link reliability. PUMA's exploratory behavior enables the discovery of globally promising paths across the vehicular topology, while PSO's exploitation component refines these solutions locally to ensure stability and minimal route breaks.

By continuously updating the routing table using the best solutions obtained through Hybrid PUMA–PSO, the VANET can dynamically adapt to environmental changes such as vehicle speed, direction, and network congestion. This approach enhances both the robustness and scalability of VANET communications compared to conventional heuristics and standalone metaheuristics.

5.2. Other Potential Applications

Beyond VANETs, the proposed hybrid algorithm can be employed in several other complex optimization domains, including:

- **Feature selection in machine learning:** selecting optimal feature subsets while reducing model complexity.
- **Image segmentation and computer vision:** optimizing segmentation thresholds and cluster centers.
- **Energy management in wireless sensor networks:** minimizing total energy consumption while maintaining network coverage.
- **Scheduling and resource allocation:** solving dynamic load-balancing problems in cloud and edge computing systems.

The hybrid design, with its balanced search capabilities and convergence reliability, makes it suitable for high-dimensional, nonlinear, and multi-objective problems where traditional optimization methods fail to perform efficiently.

5.3. Summary

The integration of Hybrid PUMA–PSO into VANET route optimization represents a promising future research direction. Its adaptability to network dynamics, combined with low computational cost, makes it a viable candidate for real-time vehicular decision systems. The next phase of this research will involve the implementation of the proposed approach within a realistic VANET simulation environment (SUMO–NS3) to quantitatively evaluate improvements in routing stability, latency, and delivery performance.

6. Future Scope

The present study enhanced the exploitation capability of the PUMA optimizer through hybridization with Particle Swarm Optimization (PSO) and verified its superior performance on standard benchmark functions. Although the proposed Hybrid PUMA–PSO model demonstrated high robustness and accuracy, several promising research directions remain open for future work.

6.1. Integration with VANET Routing

A primary direction for future research is the application of Hybrid PUMA–PSO to **route optimization in Vehicular Ad-hoc Networks (VANETs)**. The optimization objectives can be extended to multi-criteria parameters such as:

- Link stability and vehicle connectivity,
- Vehicular speed and direction variability,

- Traffic density and network congestion,
- End-to-end delay and packet delivery ratio.

The hybrid model can dynamically select optimal routes by balancing these conflicting objectives in real time. Integration with network simulators such as **SUMO** (Simulation of Urban Mobility) and **NS-3** will allow comprehensive evaluation of metrics like throughput, latency, and route reliability.

6.2. Adaptive and Intelligent Parameter Control

Another key enhancement involves incorporating adaptive control mechanisms that adjust algorithmic parameters during execution. This includes:

- Adaptive coefficients that fine-tune exploration and exploitation,
- Reinforcement learning-based agents to guide parameter updates,
- Fuzzy or self-learning strategies for balancing population diversity.

Such methods will enable Hybrid PUMA–PSO to automatically respond to changes in problem complexity and convergence behavior.

6.3. Multi-objective and Domain Extensions

Future research can also extend the current single-objective framework into a **multi-objective** setting, where competing goals such as minimizing delay, maximizing throughput, and reducing energy consumption are optimized simultaneously. In addition, Hybrid PUMA–PSO can be applied to various other domains, including:

- Energy-efficient routing in wireless sensor networks,
- Feature selection and hyperparameter optimization in machine learning,
- Task scheduling and load balancing in edge/cloud computing systems.

6.4. Theoretical and Analytical Studies

Finally, deeper theoretical analysis will be conducted to examine the convergence behavior and computational complexity of the hybrid optimizer. This includes deriving mathematical bounds on convergence rate and runtime performance. Such analysis will help formalize the algorithm's stability and scalability across problem domains.

Overall, future work will focus on deploying the Hybrid PUMA–PSO algorithm in dynamic, real-time environments—bridging the gap between simulation-based optimization and practical deployment in **intelligent transportation and communication systems**.

7. Conclusion

This study presented the development and analysis of a Hybrid PUMA–PSO metaheuristic optimizer designed to overcome the exploitation limitations of the original PUMA algorithm. The hybridization framework integrates the exploration strength of PUMA with the refinement capability of Particle Swarm Optimization, enabling the algorithm to maintain population diversity while achieving faster and more stable convergence.

Comprehensive experiments on benchmark functions (F1–F7) demonstrated that the proposed Hybrid PUMA–PSO consistently outperforms the original PUMA and several state-of-the-art algorithms, including SCA, GWO, WOA, TSA, FHO, and PSO. Statistical significance tests and convergence analyses confirmed the algorithm's ability to balance global search and local exploitation more effectively, yielding lower mean fitness values and smoother convergence behavior.

The results validate that incorporating PSO after PUMA's exploitation phase significantly enhances its capacity for local search and fine-tuning near the global optimum. This improvement establishes the Hybrid PUMA–PSO as a robust and adaptive optimizer suitable for complex, dynamic, and nonlinear optimization tasks.

In future work, the algorithm will be extended and integrated into **Vehicular Ad-hoc Networks (VANETs)** for intelligent route optimization. By dynamically optimizing parameters such as link stability, traffic density, and delay, the hybrid model can contribute to more efficient and resilient vehicular communication systems. Beyond VANETs, the proposed method also shows promise in domains such as machine learning, resource scheduling, and energy-aware networking.

Overall, the Hybrid PUMA–PSO algorithm successfully bridges the gap between exploration-oriented and exploitation-oriented metaheuristics, offering a computationally efficient and general-purpose optimization tool for real-world applications.

References

[1] B. Abdollahzadeh, N. Khodadadi, S. Barshandeh, P. Trojovský, F. S. Gharehchopogh, E.-S. M. El-Kenawy, L. Abualigah, and S. Mirjalili, "Puma optimizer (po): A novel metaheuristic optimization algorithm and its application in machine learning," *Cluster Computing*, 2023.

[2] S. Mirjalili, "Sca: A sine cosine algorithm for solving optimization problems," *Knowledge-Based Systems*, vol. 96, pp. 120–133, 2016.

[3] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014.

[4] S. Mirjalili and A. Lewis, "The whale optimization algorithm," *Advances in Engineering Software*, vol. 95, pp. 51–67, 2016.

[5] M. Kaur and D. K. Aseri, "Tunicate swarm algorithm: A new bio-inspired based metaheuristic paradigm for global optimization," *Engineering Applications of Artificial Intelligence*, vol. 90, p. 103541, 2020.

[6] M. Azizi, S. TalatAhari, and A. H. Gandomi, "Fire hawk optimizer: A novel metaheuristic algorithm," *Artificial Intelligence Review*, vol. 55, no. 7, pp. 4753–4785, 2022.

[7] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of IEEE International Conference on Neural Networks*, Perth, Australia, 1995, pp. 1942–1948.

[8] C. A. Floudas and C. E. Gounaris, "A review of recent advances in global optimization," *Journal of Global Optimization*, vol. 45, pp. 3–38, 2009.

[9] A. Törn and A. Zilinskas, *Global Optimization*. Berlin: Springer, 1989.

[10] K. E. Parsopoulos and M. N. Vrahatis, "Recent approaches to global optimization problems through particle swarm optimization," *Natural Computing*, vol. 1, pp. 235–306, 2002.

[11] H.-G. Beyer and B. Sendhoff, "Robust optimization – a comprehensive survey," *Computer Methods in Applied Mechanics and Engineering*, vol. 196, no. 33-34, pp. 3190–3218, 2007.

[12] F. S. Gharehchopogh and H. Gholizadeh, "A comprehensive survey: whale optimization algorithm and its applications," *Swarm and Evolutionary Computation*, vol. 48, pp. 1–24, 2019.

[13] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken: John Wiley & Sons, 2009.

[14] N. Khodadadi *et al.*, "Chaotic stochastic paint optimizer (cspo)," in *Proceedings of the 7th International Conference on Harmony Search and Soft Computing Applications (ICHSA 2022)*. Singapore: Springer, 2022.

[15] K. Deb, "Multi-objective optimization," in *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*, E. K. Burke and G. Kendall, Eds. Boston: Springer, 2013, pp. 403–449.

[16] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[17] P. Trojovský and M. Dehghani, "Pelican optimization algorithm: a novel nature-inspired algorithm for engineering applications," *Sensors*, vol. 22, no. 3, p. 855, 2022.

[18] E. Rashedi, H. Nezamabadi-Pour, and S. Saryazdi, "Gsa: a gravitational search algorithm," *Information Sciences*, vol. 179, no. 13, pp. 2232–2248, 2009.

[19] E.-S. M. El-kenawy *et al.*, "Al-biruni earth radius (ber) metaheuristic search optimization algorithm," *Computers & Systems Science & Engineering*, vol. 45, pp. 1917–1934, 2023.

[20] R. V. Rao, V. J. Savsani, and D. Vakharia, "Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems," *Computer-Aided Design*, vol. 43, no. 3, pp. 303–315, 2011.

[21] H. Shayanfar and F. S. Gharehchopogh, "Farmland fertility: a new metaheuristic algorithm for solving continuous optimization problems," *Applied Soft Computing*, vol. 71, pp. 728–746, 2018.

[22] R. L. Rardin, *Optimization in Operations Research*. Upper Saddle River, NJ: Prentice Hall, 1998.

[23] S. S. Rao, *Engineering Optimization: Theory and Practice*. Hoboken: John Wiley & Sons, 2019.

[24] N. Khodadadi, S. Talatahari, and A. H. Gandomi, "Anna: advanced neural network algorithm for optimisation of structures," *Proceedings of the Institution of Civil Engineers - Structures and Buildings*, 2023, https://doi.org/10.1680/jstbu.22.00083.

[25] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.

[26] S. Mirjalili, "The ant lion optimizer," *Advances in Engineering Software*, vol. 83, pp. 80–98, 2015.

[27] H. Yapici and N. Cetinkaya, "A new meta-heuristic optimizer: Pathfinder algorithm," *Applied Soft Computing*, vol. 78, pp. 545–568, 2019.

[28] D. Karaboga and B. Basturk, "A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (abc) algorithm," *Journal of Global Optimization*, vol. 39, pp. 459–471, 2007.

[29] S. Mirjalili *et al.*, "Salp swarm algorithm: a bio-inspired optimizer for engineering design problems," *Advances in Engineering Software*, vol. 114, pp. 163–191, 2017.

[30] B. Abdollahzadeh *et al.*, "Mountain gazelle optimizer: a new nature-inspired metaheuristic algorithm for global optimization problems," *Advances in Engineering Software*, vol. 174, p. 103282, 2022.

[31] M. Yazdani and F. Jolai, "Lion optimization algorithm (loa): a nature-inspired metaheuristic algorithm," *Journal of Computational Design and Engineering*, vol. 3, no. 1, pp. 24–36, 2016.

[32] B. Abdollahzadeh, F. S. Gharehchopogh, and S. Mirjalili, "Artificial gorilla troops optimizer: a new nature-inspired metaheuristic algorithm for global optimization problems," *International Journal of Intelligent Systems*, vol. 36, no. 10, pp. 5887–5958, 2021.

[33] A. Kaveh, S. Talatahari, and N. Khodadadi, "Stochastic paint optimizer: theory and application in civil engineering," *Engineering with Computers*, pp. 1–32, 2020.

[34] N. Kumar, N. Singh, and D. P. Vidyarthi, "Artificial lizard search optimization (also): a novel nature-inspired meta-heuristic algorithm," *Soft Computing*, vol. 25, no. 8, pp. 6179–6201, 2021.

[35] B. Abdollahzadeh, F. S. Gharehchopogh, and S. Mirjalili, "African vultures optimization algorithm: a new nature-inspired metaheuristic algorithm for global optimization problems," *Computers & Industrial Engineering*, vol. 158, p. 107408, 2021.

[36] A. Faramarzi *et al.*, "Marine predators algorithm: a nature-inspired metaheuristic," *Expert Systems with Applications*, vol. 152, p. 113377, 2020.

[37] J. H. Holland, "Genetic algorithms," *Scientific American*, vol. 267, no. 1, pp. 66–73, 1992.

[38] A. Cheraghalipour, M. Hajiaghaei-Keshteli, and M. M. Paydar, "Tree growth algorithm (tga): a novel approach for solving optimization problems," *Engineering Applications of Artificial Intelligence*, vol. 72, pp. 393–414, 2018.

[39] D. Tang *et al.*, "Itgo: Invasive tumor growth optimization algorithm," *Applied Soft Computing*, vol. 36, pp. 670–698, 2015.

[40] M. Dehghani *et al.*, "Coati optimization algorithm: a new bio-inspired metaheuristic algorithm for solving optimization problems," *Knowledge-Based Systems*, vol. 259, p. 110011, 2023.

[41] D. Simon, "Biogeography-based optimization," *IEEE Transactions on Evolutionary Computation*, vol. 12, no. 6, pp. 702–713, 2008.

[42] S. Kirkpatrick, C. D. G. Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[43] O. K. Erol and I. Eksin, "A new optimization method: Big bang–big crunch," *Advances in Engineering Software*, vol. 37, no. 2, pp. 106–111, 2006.

[44] R. A. Formato, "Central force optimization," *Progress in Electromagnetics Research*, vol. 77, pp. 425–491, 2007.

[45] H. Abedinpourshotorban *et al.*, "Electromagnetic field optimization: a physics-inspired metaheuristic optimization algorithm," *Swarm and Evolutionary Computation*, vol. 26, pp. 8–22, 2016.

[46] M. Dehghani, E. Trojovska, and P. Trojovský, "A new human-based metaheuristic algorithm for solving optimization problems on the base of simulation of driving training process," *Scientific Reports*, vol. 12, no. 1, p. 9924, 2022.